

Desafio Prático: Git Flow e CI na Construção de um Dashboard O11y

Objetivo: Simular o desenvolvimento de um painel de observabilidade moderno com tema "Dark Mode" e elementos neon, utilizando o fluxo de trabalho profissional Git Flow e Integração Contínua (CI).

Dinâmica do Desafio

- Este desafio deve ser executado em **dupla**. O roteiro prático possui 20 etapas.
- Cada integrante da dupla deve assumir e desenvolver **10 tarefas**.
- A execução de versionamento deve ocorrer estritamente via terminal.
- Para **cada passo** (a partir do Passo 3), o responsável deve:
 1. Inferir o prefixo correto da branch baseado nas regras do Git Flow (ex: feature/, release/, hotfix/).
 2. Criar a branch a partir da base correta (geralmente develop).
 3. Adicionar o código solicitado e fazer o commit.
 4. Abrir um Pull Request (PR) para a branch develop.
- O colega de dupla **deve obrigatoriamente** revisar o código e aprovar o PR antes da execução do merge.

Passo 1: Estrutura Base e Push Inicial

Desafio Git: Inicializar o repositório localmente, criar a base do projeto e fazer o primeiro push direto na main. Isso servirá como a fundação para a conexão com o servidor de hospedagem.

Arquivo: index.html

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width,
initial-scale=1.0">
<title>O11y Dashboard</title>
<link rel="stylesheet" href="style.css">
</head>
<body>
```

```
<div class="app-layout">
</div>
</body>
</html>
```

Arquivo: style.css

```
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
  font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
}
body {
  background-color: #0f172a;
  color: #f8fafc;
  min-height: 100vh;
}
```

Passo 2: Conexão com a Vercel (Deploy Contínuo)

Desafio: Antes de começar a abrir novas branches e criar funcionalidades, vamos garantir que o deploy contínuo está ativo. Assim, vocês poderão acompanhar a evolução visual do painel a cada aprovação de PR e merge no fluxo principal.

1. Acessem o painel da Vercel.
2. Importem o repositório Git onde fizeram o push do Passo 1.
3. Deixem as configurações de Framework Preset como "Other".
4. Cliquem em Deploy. O site estará no ar apenas com uma tela vazia no tema escuro.

Ação obrigatória: Logo após esse deploy, voltem ao terminal e criem a branch develop a partir da main e enviem para o repositório remoto. A partir de agora, o Git Flow tradicional entra em ação!

Passo 3: Pipeline de Integração Contínua (CI)

Desafio Git: Criar uma branch para implementar uma funcionalidade técnica na infraestrutura do repositório.

Ação: Construam uma pipeline de Integração Contínua (CI) utilizando o GitHub Actions ou GitLab CI. A pipeline deve ser configurada para ser acionada sempre que houver um Pull Request para as branches develop ou main, realizando uma validação para garantir que os arquivos não quebrem o fluxo. Usem o que já conhecem para estruturar este arquivo YAML. *Lembrem-se: comitar, abrir PR para develop, o colega revisa e aprova.*

Passo 4: Variáveis de Tema (Design System)

Desafio Git: Nova funcionalidade de estilos (feature branch).

Arquivo: Adicionar no topo do style.css

```
:root {  
  --bg-main: #0f172a;  
  --bg-card: #1e293b;  
  --text-main: #f8fafc;  
  --text-muted: #94a3b8;  
  --accent-blue: #38bdf8;  
  --accent-green: #4ade80;  
  --accent-red: #f87171;  
  --border-color: #334155;  
}
```

Passo 5: Grid Principal de Layout

Desafio Git: Estruturar a área de trabalho visualmente.

Arquivo: Atualizar o style.css

```
.app-layout {  
  display: grid;  
  grid-template-columns: 250px 1fr;  
  grid-template-rows: 70px 1fr;  
  height: 100vh;  
}
```

Passo 6: Container da Sidebar

Desafio Git: Criar a base do painel lateral de navegação.

Arquivo: Dentro de .app-layout no index.html

```
<aside class="sidebar">  
</aside>
```

Arquivo: style.css

```
.sidebar {  
  background-color: var(--bg-card);  
  grid-row: 1 / 3;  
  border-right: 1px solid var(--border-color);  
  padding: 1.5rem;  
  display: flex;  
  flex-direction: column;  
  gap: 2rem;  
}
```

Passo 7: Logotipo da Sidebar

Desafio Git: Adicionar a identidade visual ao projeto.

Arquivo: Dentro de `<aside class="sidebar">`

```
<div class="brand">
  <span class="brand-icon">⚡</span>
  <h1>O11y-Dash</h1>
</div>
```

Arquivo: style.css

```
.brand {
  display: flex;
  align-items: center;
  gap: 0.75rem;
}
.brand h1 {
  font-size: 1.25rem;
  color: var(--accent-blue);
  letter-spacing: 1px;
}
```

Passo 8: Menu de Navegação

Desafio Git: Adicionar os links de roteamento do sistema.

Arquivo: Abaixo da `.brand` na sidebar

```
<nav class="menu">
  <a href="#" class="active">Visão Geral</a>
  <a href="#">Métricas</a>
  <a href="#">Logs de Rede</a>
  <a href="#">Alertas</a>
  <a href="#">Configurações</a>
</nav>
```

Arquivo: style.css

```
.menu {  
  display: flex;  
  flex-direction: column;  
  gap: 0.5rem;  
}  
.menu a {  
  color: var(--text-muted);  
  text-decoration: none;  
  padding: 0.75rem 1rem;  
  border-radius: 8px;  
  transition: all 0.3s;  
}  
.menu a:hover, .menu a.active {  
  background-color: #0f172a;  
  color: var(--accent-blue);  
}
```

Passo 9: Barra Superior (Topbar)

Desafio Git: Criar o cabeçalho superior do layout.

Arquivo: Dentro de .app-layout, abaixo da sidebar

```
<header class="topbar">  
</header>
```

Arquivo: style.css

```
.topbar {  
  background-color: var(--bg-card);  
  border-bottom: 1px solid var(--border-color);  
  display: flex;  
  align-items: center;  
  justify-content: space-between;  
  padding: 0 2rem;}
```

Passo 10: Elementos da Topbar

Desafio Git: Campo de busca e ícone de perfil de usuário.

Arquivo: Dentro de `<header class="topbar">`

```
<div class="search-box">
  <input type="text" placeholder="Buscar métricas, nós ou logs...">
</div>
<div class="user-profile">
  <div class="avatar">Admin</div>
</div>
```

Arquivo: style.css

```
.search-box input {
  background-color: var(--bg-main);
  border: 1px solid var(--border-color);
  color: var(--text-main);
  padding: 0.5rem 1rem;
  border-radius: 6px;
  width: 300px;
}
.search-box input:focus {
  outline: 1px solid var(--accent-blue);
}
.avatar {
  background-color: var(--accent-blue);
  color: #000;
  padding: 0.5rem 1rem;
  border-radius: 20px;
  font-weight: bold;
}
```

Passo 11: Container de Conteúdo Principal

Desafio Git: Estruturar o painel que receberá os gráficos.

Arquivo: Dentro de .app-layout, abaixo da topbar

```
<main class="content-area">
  <div class="dashboard-header">
    <h2>Visão Geral do Sistema</h2>
    <span class="status-badge">● Sistema Operacional</span>
  </div>
</main>
```

Arquivo: style.css

```
.content-area {
  padding: 2rem;
  overflow-y: auto;
}
.dashboard-header {
  display: flex;
  justify-content: space-between;
  align-items: center;
  margin-bottom: 2rem;
}
.status-badge {
  background-color: rgba(74, 222, 128, 0.1);
  color: var(--accent-green);
  padding: 0.5rem 1rem;
  border-radius: 20px;
  font-size: 0.875rem;
}
```


Passo 12: Grid de KPIs

Desafio Git: Criar a estrutura base que alinhará os cartões numéricos.

Arquivo: Dentro de `<main class="content-area">`, após o header

```
<div class="kpi-grid">
</div>
```

Arquivo: style.css

```
.kpi-grid {
  display: grid;
  grid-template-columns: repeat(4, 1fr);
  gap: 1.5rem;
  margin-bottom: 2rem;
}

.kpi-card {
  background-color: var(--bg-card);
  padding: 1.5rem;
  border-radius: 12px;
  border: 1px solid var(--border-color);
}
```

Passo 13: Cartão de Uptime

Desafio Git: Adicionar o primeiro card de métrica.

Arquivo: Dentro da `.kpi-grid`

```
<div class="kpi-card">
  <h3>Uptime Servidores</h3>
  <div class="value">99.99%</div>
  <div class="trend positive">↑ 0.01%</div>
</div>
```

Arquivo: style.css

```
.kpi-card h3 {
  color: var(--text-muted);
  font-size: 0.875rem;
  text-transform: uppercase;
}
.kpi-card .value {
  font-size: 2rem;
  font-weight: bold;
  margin: 0.5rem 0;
}
.trend.positive {
  color: var(--accent-green);
  font-size: 0.875rem;
}
```

Passo 14: Cartão de Latência

Desafio Git: Adicionar métrica de performance da rede.

Arquivo: Dentro da .kpi-grid

```
<div class="kpi-card">
  <h3>Latência Média (p95)</h3>
  <div class="value">42ms</div>
  <div class="trend negative">↓ 5ms</div>
</div>
```

Arquivo: style.css

```
.trend.negative {
  color: var(--accent-blue);
  font-size: 0.875rem;
}
```

Passo 15: Cartão de Requisições

Desafio Git: Adicionar card indicando o volume de tráfego.

Arquivo: Dentro da .kpi-grid

```
<div class="kpi-card">
  <h3>Requisições / seg</h3>
  <div class="value">12,450</div>
  <div class="trend positive">↑ 1,200</div>
</div>
```

Passo 16: Cartão de Taxa de Erro

Desafio Git: Adicionar card de alerta para métricas de falha.

Arquivo: Dentro da .kpi-grid

```
<div class="kpi-card">
  <h3>Taxa de Erro (5xx)</h3>
  <div class="value alert">0.15%</div>
  <div class="trend danger">↑ 0.05%</div>
</div>
```

Arquivo: style.css

```
.kpi-card .value.alert {
  color: var(--accent-red);
}
.trend.danger {
  color: var(--accent-red);
  font-size: 0.875rem;
}
```

Passo 17: Área do Gráfico Principal

Desafio Git: Criar um mock visual de um gráfico em barras indicando o uso de CPU e rede.

Arquivo: Abaixo da .kpi-grid

```
<div class="chart-section">
  <h3>Tráfego de Rede vs CPU</h3>
  <div class="chart-mock">
    <div class="bar" style="height: 60%"></div>
    <div class="bar" style="height: 80%"></div>
    <div class="bar" style="height: 40%"></div>
    <div class="bar" style="height: 90%"></div>
    <div class="bar" style="height: 50%"></div>
    <div class="bar" style="height: 70%"></div>
    <div class="bar" style="height: 30%"></div>
  </div>
</div>
```

Arquivo: style.css

```
.chart-section {
  background-color: var(--bg-card);
  padding: 1.5rem;
  border-radius: 12px;
  border: 1px solid var(--border-color);
  margin-bottom: 2rem;
}

.chart-mock {
  height: 200px;
  display: flex;
  align-items: flex-end;
  gap: 10px;
  margin-top: 1rem;
}

.bar {
  flex-grow: 1;
  background: linear-gradient(to top, var(--accent-blue),
transparent);
```

```
border-top: 2px solid var(--accent-blue);
border-radius: 4px 4px 0 0;
transition: height 0.5s;
}
```

Passo 18: Seção de Logs Recentes

Desafio Git: Criar a lista de eventos e popular com logs simulados para ver a paleta de cores em ação.

Arquivo: Abaixo da `.chart-section` no `index.html`

```
<div class="logs-section">
  <h3>Logs de Eventos Recentes</h3>
  <div class="log-list">
    <div class="log-item error">
      <span class="timestamp">10:45:21</span>
      <span class="service">[auth-service]</span>
      <span class="message">Connection timeout to database</span>
    </div>
    <div class="log-item info">
      <span class="timestamp">10:45:19</span>
      <span class="service">[payment-api]</span>
      <span class="message">Healthcheck passed</span>
    </div>
  </div>
</div>
```

Arquivo: Adicionar ao `style.css`

```
.logs-section {
  background-color: var(--bg-card);
  padding: 1.5rem;
  border-radius: 12px;
  border: 1px solid var(--border-color);
}
.log-list {
  margin-top: 1rem;
```

```

display: flex;
flex-direction: column;
gap: 0.5rem;
}
.log-item {
display: grid;
grid-template-columns: 100px 150px 1fr;
padding: 0.75rem;
border-radius: 6px;
font-family: monospace;
font-size: 0.875rem;
background-color: var(--bg-main);
}
.log-item.error { border-left: 4px solid var(--accent-red); }
.log-item.info { border-left: 4px solid var(--accent-green); }
.timestamp { color: var(--text-muted); }
.service { color: var(--accent-blue); }

```

Passo 19: Preparação para Lançamento (Release)

Desafio Git: Criar uma branch de release/ focada no empacotamento da versão (v1.0.0) e ajustes finais de UI.

Arquivo: Atualizar a tag title no index.html

```
<title>Oilly Dashboard - v1.0.0</title>
```

Arquivo: Estilizar a barra de rolagem no final do style.css

```

::-webkit-scrollbar {
width: 8px;
}
::-webkit-scrollbar-track {
background: var(--bg-main);
}
::-webkit-scrollbar-thumb {
background: var(--border-color);
border-radius: 4px;
}

```

```
}
```

Ação: O fluxo do Git Flow pede o merge dessa branch **tanto na main quanto na develop**. Como o projeto está conectado à Vercel, o merge na main disparará o deploy da versão final!

Passo 20: Correção Crítica em Produção (Hotfix)

Desafio Git: Simular um erro urgente reportado após o deploy. O layout quebra em telas de dispositivos móveis. A dupla deve criar uma branch de hotfix/ diretamente a partir da main.

Arquivo: Adicionar os ajustes responsivos ao final do style.css

```
@media (max-width: 1024px) {  
  .kpi-grid {  
    grid-template-columns: repeat(2, 1fr);  
  }  
}  
  
@media (max-width: 768px) {  
  .app-layout {  
    grid-template-columns: 1fr;  
    grid-template-rows: auto 70px 1fr;  
  }  
  .sidebar {  
    display: none;  
  }  
  .kpi-grid {  
    grid-template-columns: 1fr;  
  }  
}
```

Ação: Após o commit, façam o merge imediato na main e também devolvam para a develop para garantir que o erro não volte nas próximas atualizações. A Vercel subirá o hotfix ao vivo instantaneamente.